

Moore to the rescue

Hollywood actress Demi Moore saved a distressed teenager from committing suicide, by raising an alarm on Twitter

Judgement

Twitter users, aka tweeple, declared Australian PM Kevin Rudd the winner in the nation's healthcare debate



These days you can't avoid Death, Taxes and Twitter

Nishith Rastogi

readersletters@thinkdigit.com

Twitter is a microblogging service that lets you update your status in 140 character or less. In recent times, it has become an influential social network and learning its API is a must for any web developer aiming to have a full arsenal. Here we present a quick overview of the same.

For the uninitiated, API stands for "Application Programming Interface", typically providing a way for third-party developers to interact with a software product or web service. For Twitter, there is an official API available that provides a 'method' for every feature that is available via the original Twitter web site, including, but not limited to, posting, retrieving, sorting, searching and the like. The HTTP protocol is used by programs to talk to the Twitter API.

The Twitter API is divided into two primary parts: the "Rest API" and the "Search API". In very simple terms, everything but search is handled by the Rest API (such as posting a tweet and adding friends) and the Search API is the dedicated interface for searching.

Typically, there are libraries available for using the API with various languages that save you time and effort. We'll be using the Python-Twitter wrapper over the Twitter API.

Your first call

Fire up your Python shell, or open up your favourite Python IDE and let the code flow. Our first step would be to create an instance of the `twitter.Api`

class. This can be done in two ways, one with authentication (which will effectively allow you to use the Rest API methods) and the other being without Authentication (for pure searching and status retrieval purpose).

```
>>> import twitter
>>> digit = twitter.Api()
[With Authentication]
>>> import twitter
>>> digit = twitter.Api(username='Your_Twitter_
```

```
subiet@Cub9: ~$ python
Python 2.6.4 (r264:75706, Dec 7 2009, 18:45:15)
[GCC 4.4.1] on linux2
Type "help", "copyright", "credits" or "license()" for more information.
>>> import twitter
>>> digit = twitter.Api(username='digit_subiet', password='password')
>>>
```

No, the password is not 'password' for the 'digit_subiet' account

```
Username', password='Your_Twitter_Password')
```

The first line imports the Twitter library into the environment (the Python-Twitter wrapper) and the second creates an instance called "Digit" of the `twitter.Api` class.

Class names are case sensitive and `twitter.Api` will result in an error. Another caution to exercise is to not name your file `Twitter.py` in case you are using an IDE or using a file to batch process, because in that case while doing a `import Twitter` it will give an error.

Once in, what next

Now that we have logged in, let us see what fun stuff can we do. Starting with the obvious, let us post a Hello World! tweet to announce our arrival.

```
>>> status = digit.PostUpdate('Hello World!')
```

Notice how we are storing the return value of `PostUpdate` in `status`. `status` is actually a class with the following properties. Other classes are `DirectMes-`

I R Twogrammer

Twitter has taken the world by storm. Learn how you could get hold of the mileage and create apps for the largest microblogging site on the internet

```
sage and User, which are detailed later.
status.created_at
status.created_at_in_seconds
#read only
status.favorited
status.in_reply_to_screen_name
status.in_reply_to_user_id
status.in_reply_to_status_id
status.truncated
status.source
status.id
status.text
status.relative_created_at
#read only
status.user
```

If you want to post a tweet and then print the name of the user who updated it in your console it will be

```
>>> status = digit.PostUpdate('Hello World!')
>>> print status.user.name
```

(it is `user.name` since `user` itself is a class with multitude of properties including `user_id`, `profile_image_url`, etc and so on).

The `status.id` field is very important, as it used as the Unique Identifier for the tweet, in case you want to manipulate it later like mark it as favourite or delete it.

So the code for posting something and then effectively destroying it would be

```
>>> status = digit.PostUpdate('Hello World!')
>>> digit.DestroyStatus(status.id)
```

Let us now see how we can get the timeline of any Twitter user

```
>>> st1 = digit.GetUserTimeline('Twitter_handle', No_of_Status)
to print them, you can do the following
>>> for s in st1 :
...     print s.user.name,
...     "(" , s.user.screen_name, ")" : "
...     print s.text
```